

# Billing and Insurance Extensions to the FEN Schema: Patient-Facing Improvements for Bills, Denials, and Coverage

Phoros Engineering

July 2026

## 1 Overview

This document specifies extensions to the billing and insurance facts of the FEN schema. The current schema captures a workable claims spine: coverage enrollment, claim submission, adjudication, payment, pre-authorization requests and decisions, and appeals. These extensions address gaps that prevent the schema from supporting patient-facing features such as bill reconciliation, denial triage, deductible tracking, and structured appeals.

The changes are motivated by patient value rather than by any particular go-to-market strategy. A patient who wants to detect a duplicate charge on a provider bill, understand why a prior authorization was denied, see how close they are to their deductible, or track an appeal through its lifecycle cannot do so with the current schema. The extensions here close those gaps.

Section 2 introduces an **Institution** entity to replace bare strings for payers, providers, pharmacies, and other organizations. Section 3 introduces a **ProviderStatement** payload distinct from **Claim** and **Adjudication**. Section 4 makes claim service lines first-class. Section 5 adds **PlanAccumulator** as an event-driven observation series. Section 6 converts **Coverage** from a snapshot to a timeline and introduces a **PlanStructure** type distinct from **CoverageType**. Section 7 replaces the free-text denial reason with structured CARC/RARC codes. Section 8 adds an **AppealEvent** pattern and discusses the event-driven vs. state-machine design tradeoff. Section 9 adds a **PharmacyClaim** payload and a structured **PharmacyCoverageResult**. It also represents a point-of-prescribing formulary check as its own **Fact**. Section 10 expands **PayerType** and revises **PaymentPlan** to distinguish provider-managed plans from third-party medical financing, collections agencies, and charity programs. Section 11 adds a **FactRelation** entity for typed connections between billing facts.

All extensions preserve the graph-oriented representation of the FEN schema. Entities remain nodes, and cross-fact relations are represented through typed relations and **FactId** references. The extensions use the core **Fact** distinction between assertion provenance and evidence: **Fact.assertion** records who asserted a billing **Fact**, when, and how; **Fact.evidence** records the source citations and prior **Facts** that support it.

### 1.1 From a billing document to FEN Facts

Consider a payer Explanation of Benefits (EOB) received as a PDF. The PDF is not itself a **Fact**. It is preserved outside FEN as an immutable **SourceArtifact**, whose bytes are held by a **StoredObject**. A citation to the relevant page or region becomes a **SourceCitationRef**. Normalization may then produce **Claim** and **Adjudication** **Facts** that cite that location as evidence. The payer's claim number is an **ExternalRef**: it identifies the source record, but does not by itself

support the normalized assertion. A typed `FactRelation` may connect the Claim to the Adjudication. Processing state and patient-facing reconciliation views remain `FactOperational` data or projections.

The same boundary applies to provider statements, remittance files, payer API responses, pharmacy transactions, and Epic EHI table rows:

| Concern                                   | Representation                                                      |
|-------------------------------------------|---------------------------------------------------------------------|
| Original bytes or archive                 | <code>SourceArtifact</code> $\rightarrow$ <code>StoredObject</code> |
| Exact supporting passage or row           | <code>SourceCitationRef</code>                                      |
| Who asserted the normalized claim         | <code>AssertionProvenance</code>                                    |
| Supporting source material or prior Facts | <code>EvidenceRef</code>                                            |
| Source-system claim or statement identity | <code>ExternalRef</code>                                            |
| Relationship between billing Facts        | typed <code>FactRelation</code>                                     |
| Transient processing or display state     | <code>FactOperational</code> or projection                          |

## 2 Institution as a first-class entity

The current schema represents payers, providers, pharmacies, and other organizations as bare `String` fields:

```
Coverage    { payer: String, plan: String, ... }
Claim       { payer: String, ... }
PreAuthRequest { payer: String, ... }
Encounter   { facility: Option<String>, ... }
```

Bare strings have three problems. They admit no identity resolution: “AHN,” “Allegheny Health Network,” and “Allegheny Health Network, Inc.” are three distinct string values but one institution. They carry no canonical identifier such as an NPI (National Provider Identifier), a Payer ID, or an NCPDP (pharmacy) number, so external correlation is impossible. They prevent cross-fact reasoning: forty-seven claims from the same payer appear as forty-seven distinct string values rather than forty-seven references to one entity.

An `Institution` entity resolves this:

```
Institution {
  id: InstitutionId,
  display_name: String,
  aliases: Vec<String>,
  canonical_ids: Vec<CanonicalId>,
  institution_type: InstitutionType,
}

CanonicalId {
  Npi(String), // National Provider Identifier
  PayerId(String), // CMS payer identifier
  Ncpdp(String), // pharmacy
  TaxId(String), // EIN
  ClearinghouseId { system: String, value: String },
  Other { system: String, value: String },
}
```

```

}

InstitutionType {
    Payer,
    Provider,
    Pharmacy,
    Laboratory,
    ImagingCenter,
    Employer,
    ThirdPartyAdministrator,
    Other,
}

```

**Institution** is contextual identity data, not a fourth FEN epistemic layer. It is neither a Fact, a ProblemEpisode, nor a Narrative. Billing Facts refer to a stable **InstitutionId**; the registry provides canonical identity and display information. A registry merge or alias decision must be authored and auditable, but it must not rewrite the payer or provider name preserved in the original source artifact.

Facts reference institutions by **InstitutionId**, and the institution registry holds display names, aliases, and canonical identifiers. This is analogous to **SubjectId**: a stable internal identifier that resolves to external identity systems.

The identity resolution problem is real but bounded: identity resolution runs at ingestion, not at query time, and produces an authored decision. When two ingestion sources refer to the same institution differently, the resolver produces a mapping and records the decision as an **InstitutionAliasRelation**. Ambiguous cases surface for review rather than silently merging or duplicating.

Fields that previously carried bare institution strings are migrated to **InstitutionId** references:

```

Coverage {
    payer: InstitutionId,
    plan:      String,
    // ...
}

Claim {
    payer: InstitutionId,
    // ...
}

PreAuthRequest {
    payer: InstitutionId,
    // ...
}

```

Clinical facts such as **Encounter.facility** may continue to use bare strings in the near term, since the reasoning value of institution identity is highest for insurance-related facts. Migration of clinical facts to **InstitutionId** is a follow-up, not a prerequisite.

### 3 ProviderStatement

A **ProviderStatement** represents the bill a provider sends to the patient. It is distinct from a **Claim** (what the provider billed the payer) and from an **Adjudication** (what the payer decided). The provider bill and the payer's Explanation of Benefits arrive on different timelines, use different formats, and routinely disagree. A patient detects billing errors by comparing the two. The schema must represent both.

```
FactPayload {
  // ... existing variants ...

  ProviderStatement {
    statement_id: String,
    provider: InstitutionId,
    statement_date: Date,
    due_date: Option<Date>,
    service_period: TemporalAnchor,
    lines: Vec<StatementLine>,
    total_charges: Money,
    insurance_paid: Money,
    adjustments: Money,
    prior_balance: Money,
    amount_due: Money,
    related_claim: Option<FactId>,
  },
}
```

The **related\_claim** field links to the **Claim** fact this statement corresponds to, when the correspondence can be established. A missing link is itself informative: it means the patient has received a bill for a service that has no matching claim in their record, which often indicates the provider has not yet billed insurance.

```
StatementLine {
  service_date: Date,
  description: String,
  procedure_code: Option<CodedValue>, // CPT or HCPCS
  units: u32,
  charge: Money,
  insurance_payment: Money,
  adjustment: Money,
  patient_responsibility: Money,
}
```

An EOB received as a PDF or 835 transaction is preserved as a **SourceArtifact**; it is not recoverable merely by re-rendering the normalized Adjudication Facts. A later **ExplanationOfBenefits** Fact payload would be justified only if it captured an EOB-level domain assertion, not simply because the original file needs a container. Either way, the original artifact and the citations used during normalization remain intact.

## 4 Claim service lines as first-class structure

The current `Claim` payload treats billed codes and billed amounts as aggregate:

```
Claim {
  claim_id:      String,
  billed_codes:   Vec<CodedValue>,
  billed_amount:  Money,
  payer:         String,
}
```

Real claims have multiple service lines, each with its own procedure code, unit count, billed amount, and adjudication outcome. To detect duplicate charges (the same code billed twice), unbundling violations (services billed separately that should have been billed together), or upcoding (a higher-severity code than the encounter warranted), a claim must expose its lines. Similarly, denials frequently occur at the line level: a claim as a whole may be marked `PartiallyPaid` while one line is denied and the others are paid.

```
Claim {
  claim_id:      String,
  lines:         Vec<ClaimLine>,
  total_billed:  Money, // sum of line.billed_amount
  payer:         InstitutionId,
  claim_type:    ClaimType, // Professional | Institutional
  service_period: TemporalAnchor,
}
```

The aggregate `billed_codes` field is removed; consumers derive the code list from `lines`. The aggregate `billed_amount` is renamed `total_billed` and constrained to equal the sum of line-level billed amounts. Ingestion validates this invariant.

```
ClaimLine {
  line_number:  u32,
  service_date: Date,
  procedure_code: CodedValue, // CPT or HCPCS
  modifiers:    Vec<CodedValue>, // CPT modifiers
  diagnosis_refs: Vec<CodedValue>, // ICD-10 pointers
  units:        u32,
  billed_amount: Money,
  place_of_service: Option<CodedValue>,
  rendering_provider: Option<String>,
}
```

Adjudication correspondingly gains line-level detail:

```
Adjudication {
  claim_id:      String,
  lines:         Vec<AdjudicationLine>,
  total_allowed: Money,
  total_patient_resp: Money,
  status:        AdjudicationStatus,
  processed_date: Date,
```

```

}

AdjudicationLine {
  line_number:  u32, // matches ClaimLine.line_number
  allowed_amount:  Money,
  paid_amount:  Money,
  patient_responsibility:  Money,
  deductible_applied:  Money,
  coinsurance_applied:  Money,
  copay_applied:  Money,
  denial_codes:  Vec<CodedValue>, // CARC/RARC per Sec. 7
  line_status:  AdjudicationStatus,
}

```

## 5 PlanAccumulator

Deductible and out-of-pocket accumulator state is derived from adjudications but reported by the payer as a snapshot. Patients need to see it directly. Providers use it to quote patient responsibility before a service. The current schema has no representation of accumulator state.

```

FactPayload {
  // ...

  PlanAccumulator {
    coverage_ref:  FactId, // link to Coverage fact
    as_of:  Date,
    plan_year:  u16,
    individual_deductible:  AccumulatorAmount,
    family_deductible:  Option<AccumulatorAmount>,
    individual_oop_max:  AccumulatorAmount,
    family_oop_max:  Option<AccumulatorAmount>,
    network_scope:  NetworkScope,
    reported_via:  Option<AccumulatorChannel>,
  },
}

AccumulatorAmount {
  limit:  Money,
  met:  Money, // amount accumulated toward the limit
}

NetworkScope { InNetwork, OutOfNetwork, Combined }

AccumulatorChannel {
  PayerApi, // 270/271 eligibility or Patient Access API
  ProviderPortal, // captured at time of service
  PatientStatement, // from paper or portal EOB
}

```

`reported_via` describes a domain communication channel; it is not evidence provenance. The enclosing Fact cites the response, statement, or table row through `Fact.evidence`. A value com-

puted from prior Adjudication Facts instead uses `AssertionMethod::Inferred`, cites those Facts through `EvidenceRef::PriorFact`, and records the evaluator or rule version used.

**Design decision: event-driven, not superseded.** Each `PlanAccumulator` fact is an independent observation, not a correction of a prior one. A new accumulator reading does not supersede the previous one; both facts remain `Active` and reflect the accumulator state at their respective `as_of` dates. This diverges from an earlier design that used `FactStatus::Superseded` with an `AccumulatorUpdate` supersession reason.

The rationale: supersession is appropriate when a fact is being corrected or refined — the earlier fact was wrong or partial, and the newer fact replaces it. Accumulator readings are not corrections. Each reading is a true observation at a real point in time, and the progression of readings is itself informative. A patient who hit their deductible at three different visits wants to see the progression (“\$500 met in March, \$1,200 met in May, \$3,000 met in July”), not just the most recent snapshot. Supersession would erase the progression.

This mirrors how `LabResult` works in the existing schema. Every lab draw is a separate `LabResult` fact with its own timestamp, not a supersession of the previous draw. A hemoglobin value from last month is not made wrong by a hemoglobin value from this month; both are true observations at their respective times. The same reasoning applies to accumulators.

The read model selects the most recent `PlanAccumulator` for the relevant coverage and plan year. A historical query such as “what was my deductible progress on July 1?” selects the latest observation available at that date. Neither query requires supersession.

Supersession is still available for the rare case where a payer retroactively corrects an accumulator reading. The prior Fact is then marked `Superseded` for clinical refinement. This treats the change as a correction of one specific observation rather than as a routine update.

## 6 Coverage timeline

The current `Coverage` payload represents coverage as a fact without explicit temporal bounds:

```
Coverage {
  payer:      String,
  plan:       String,
  member_id:  String,
  group_id:   Option<String>,
  coverage_type: CoverageType,
}
```

Coverage changes: patients change employers, plans renew, dependents are added, secondary coverage begins. Coverage must be represented as a period, not a point. The `Fact.occurred_at` field can carry a `TemporalAnchor::Period`, but the schema does not currently express the domain-specific fields that make coverage-as-timeline usable.

```
Coverage {
  payer:  InstitutionId,
  plan:   String,
  member_id: String,
  group_id: Option<String>,
  coverage_type: CoverageType,
  plan_structure: PlanStructure,
  effective_from: Date,
```

```

    effective_through: Option<Date>, // None means currently active
    plan_year: Option<u16>,
    network_status: NetworkStatus,
    relationship_to_subscriber: SubscriberRelationship,
    policy_holder: Option<String>,
}

NetworkStatus { InNetwork, OutOfNetwork, Unknown }

SubscriberRelationship {
    Self_, // patient is the subscriber
    Spouse,
    Dependent,
    Other,
}

```

**effective\_through:** `None` indicates the coverage is currently active. When coverage ends, the fact is superseded and a new **Coverage** fact is created for the replacement coverage.

The **plan\_year** field distinguishes “same employer, different plan year” (a plan renewal with updated benefits) from “different employer.” This distinction matters for accumulator resets: at the start of a new plan year the deductible resets even if the coverage is continuous.

## 6.1 PlanStructure

The existing **CoverageType** enum (**Commercial** | **Medicare** | **Medicaid** | **Tricare** | **SelfPay** | **Other**) distinguishes *who pays* but not *how the plan is structured*. A commercial HDHP with a \$6,000 deductible and 20% coinsurance behaves totally differently from a commercial PPO with \$30 copays and no deductible, and the patient’s cost-share reasoning depends on the structure. **PlanStructure** captures the structural distinction:

```

PlanStructure {
    plan_type: PlanType,
    has_deductible: bool,
    has_coinsurance: bool,
    has_copay: bool,
    is_hsa_eligible: bool, // HDHP paired with HSA
}

PlanType {
    Hmo,
    Ppo,
    Epo,
    Pos,
    Hdhp, // High-Deductible Health Plan
    Indemnity,
    Unknown,
}

```

**PlanStructure** is deliberately minimal. It does not attempt to model the full plan design (formulary, service-level coverage rules, network tiers, benefit-year rules). Those belong in a separate plan-design contextual model outside **FactPayload**. The structural type is included on **Coverage** because the patient needs to know whether their next visit will hit a deductible or a copay, and

that requires knowing the plan structure.

`PlanStructure` is a field on `Coverage` rather than a separate fact. It describes the coverage and does not vary independently over time — when the plan structure changes (e.g., the employer switches plans mid-year), a new `Coverage` fact is created and the prior one is superseded.

## 7 Structured denial reasons

The current `PreAuthDecision` payload uses a free-text denial reason:

```
PreAuthDecision {
  authorization_id: String,
  status: AuthorizationStatus,
  reason: Option<String>,
  approved_service: Option<CodedValue>,
  valid_through: Option<Date>,
}
```

Free-text denial reasons are unparseable. Payers actually communicate denials using standardized codes: CARC (Claim Adjustment Reason Codes, maintained by X12) and RARC (Remittance Advice Remark Codes, maintained by CMS). Structured codes enable a real feature: given a denial reason, determine whether the patient has a viable appeal path, what documentation is required, and what the timeline is.

```
PreAuthDecision {
  authorization_id: String,
  status: AuthorizationStatus,
  denial_codes: Vec<CodedValue>, // CARC and RARC
  denial_narrative: Option<String>, // free-text supplement
  approved_service: Option<CodedValue>,
  valid_through: Option<Date>,
  decision_date: Date,
  appeal_deadline: Option<Date>,
}
```

The same treatment applies to `AdjudicationLine.denial_codes` (see Section 4). CARC and RARC vocabularies are represented as `CodedValue` with the code system identifier distinguishing the two.

The free-text `denial_narrative` field is preserved as an optional supplement. Payer notices frequently include narrative explanations that add context beyond the structured code (“member has not tried step-therapy tier 1 medication” is a RARC N123 with a specific meaning; the narrative provides the humanized version).

When a payer supplies the codes, the normalized Fact cites the relevant notice or transaction location. When Phoros maps free text to codes, that mapping is a separate inferred coding assertion supported by the original denial Fact. It does not silently convert model output into payer-authored content.

## 8 Appeal lifecycle

The current `Appeal` payload captures the case but not its progression:

```
Appeal {
  original_decision_id: FactId,
  grounds: String,
  supporting_facts: Vec<FactId>,
}
```

An appeal is a multi-week process with distinct states: filed, acknowledged, additional information requested, decided at first level, escalated to second level, escalated to external review, resolved. Without lifecycle tracking, the schema cannot answer basic questions such as “is this appeal still open,” “did it succeed,” or “what level is it at.”

### 8.1 Design decision: event-driven vs. state-machine

Two designs were considered for appeal lifecycle. The adopted design uses the same event pattern as medication modeling: an **Appeal** Fact establishes the case, and **AppealEvent** Facts record each state transition. The current state of an appeal is computed by folding over its events. The state-machine design (not adopted) would place a mutable **status** field on **Appeal** that gets updated as the appeal progresses, without separate event facts.

**Arguments for event-driven (adopted).** Consistent with the medication-event idiom, so readers of the schema encounter one pattern rather than two. Each **AppealEvent** preserves its own assertion metadata and evidence: who reported the event, when and how it was asserted, and which document or prior Fact supports it. The full history is preserved as an immutable sequence of facts rather than compressed into a single mutable status. Payer decisions, patient escalations, and external review outcomes are genuine events with external actors, dates, and documents, and treating them as first-class facts respects that. The read model for “current state” is trivially implementable and cacheable.

**Arguments for state-machine (not adopted).** Simpler for common queries such as “list all open appeals” — a single field lookup rather than a fold over events. Fewer total facts. The FEN schema already has a state-machine mechanism in **FactStatus** (**Active**, **Superseded**, **EnteredInError**), so appeal state could be modeled the same way rather than introducing a second pattern.

**Why event-driven wins here.** An **AppealEvent** corresponds to a specific real-world communication: a letter from the payer acknowledging the appeal, a request for additional documentation, a decision letter with a determination, a notice of escalation to external review. The appeal itself is a case; the events are what came in the mail. Treating those communications as first-class facts (with dates, senders, assertion metadata, and evidence) is faithful to how the appeal actually unfolds. A mutable **status** field cannot carry that structure without either losing information or duplicating it in a shadow event log.

The state-machine design is preferable when the entity being tracked is genuinely a single object whose state changes internally, and the history of state changes is not itself an object of interest. Neither condition holds for appeals.

The same reasoning was applied to **PlanAccumulator** (Section 5), where each observation is a first-class event and supersession was rejected.

## 8.2 Schema

The pattern mirrors medication-order event modeling. The `Appeal` Fact establishes the case; `AppealEvent` Facts track state transitions.

```
Appeal {
  appeal_id: Option<String>, // source-system ID
  original_decision_id: FactId,
  appeal_level: AppealLevel,
  grounds: String,
  filed_date: Date,
  filed_by: AppealFiler,
}

AppealLevel {
  Internal1, // first-level internal payer review
  Internal2, // second-level internal review
  ExternalReview, // independent external review
  StateComplaint, // filed with state insurance commissioner
}

AppealFiler { Patient, Provider, AuthorizedRepresentative }

AppealEvent {
  appeal_ref: FactId, // links to the Appeal fact
  event_type: AppealEventType,
  notes: Option<String>,
},

AppealEventType {
  Acknowledged { acknowledged_date: Date },
  InfoRequested { items: Vec<String>, deadline: Date },
  InfoSubmitted { items: Vec<FactId> },
  Decided {
    outcome: AppealOutcome,
    decision_date: Date,
    denial_codes: Vec<CodedValue>, // if upheld
  },
  Escalated { to_level: AppealLevel },
  Withdrawn { withdrawn_date: Date, reason: Option<String> },
}

AppealOutcome { Overturned, PartiallyOverturned, Upheld }
```

The enclosing `Appeal` and `AppealEvent` Facts use `Fact.evidence` for supporting documents and prior Facts. A document is cited with `EvidenceRef::Source`; a prior clinical or billing Fact is cited with `EvidenceRef::PriorFact`. The `InfoSubmitted.items` field remains because it describes the content of that event—which items were submitted—rather than the epistemic support for the event record itself.

Both `Appeal` and `AppealEvent` become `FactPayload` variants. Corresponding `FactType` variants are added: `FactType::AppealEvent`.

## 9 PharmacyClaim

The current `Claim` payload is designed for medical claims (CPT/HCPSC codes, professional or institutional claim types). Pharmacy claims use different vocabularies and pricing structures: NDC codes for drug identification, days-supply and quantity dispensed, DAW (Dispense As Written) codes, formulary tier, and PBM adjudication that includes rebates and MAC (Maximum Allowable Cost) pricing. Forcing pharmacy claims into the medical `Claim` structure loses information and prevents useful features (formulary alternative suggestions, generic substitution tracking, PBM cost comparison).

A separate `PharmacyClaim` payload:

```
FactPayload {
  // ...

  PharmacyClaim {
    rx_claim_id: String,
    medication_order_ref: Option<FactId>, // link to medication-order Fact
    ndc: CodedValue, // National Drug Code
    drug_name: String,
    days_supply: u32,
    quantity_dispensed: Decimal,
    unit: CodedValue,
    dispense_date: Date,
    pharmacy: InstitutionId,
    prescriber: String,
    daw_code: Option<u8>,
    payer: InstitutionId,
    pbm: Option<InstitutionId>,
    billed_amount: Money,
    plan_paid: Money,
    patient_paid: Money,
    coverage_result: PharmacyCoverageResult,
  },
}
```

The `medication_order_ref` field links the pharmacy claim back to the originating medication-order Fact when available. This link is essential for reasoning about medication adherence (comparing prescribed days-supply to actual dispensing intervals) and for detecting formulary switches (the pharmacy dispensed a different NDC than prescribed).

### 9.1 PharmacyCoverageResult

An earlier design placed covered, not-covered, prior-authorization, step-therapy, quantity-limit, and cash-pay outcomes in one status enum. In practice pharmacy rejections often carry multiple reasons simultaneously (“not covered on formulary, and also requires step therapy, and also has a quantity limit”), which an enum with mutually exclusive variants cannot represent. A struct that composes the individual conditions handles this better:

```
PharmacyCoverageResult {
  outcome: CoverageOutcome,
  tier: Option<u8>,
  prior_auth_required: bool,
```

```

    step_therapy_required: bool,
    quantity_limit_hit: bool,
    suggested_alternative: Option<CodedValue>, // NDC of covered alt
    rejection_codes: Vec<CodedValue>, // NCPDP reject codes
    requires_manufacturer_assistance: bool,
}

CoverageOutcome {
    Paid,
    Rejected,
    Pending,
    CashPay, // adjudicated as cash by patient choice or plan denial
}

```

## 9.2 MedicationCoverageCheck at prescribing

Formulary tier at the point of prescribing is separately useful from formulary tier at adjudication. Provider EHRs increasingly run real-time benefit checks (RTBC) via NCPDP formulary APIs at the moment a prescription is written, so the tier and prior-auth requirements are known before the patient ever reaches the pharmacy.

The RTBC result must not be inserted into the clinician-authored medication order as though the clinician asserted it. The PBM or benefit service made a separate assertion, often from a separate response artifact. The adopted representation is therefore another Fact:

```

FactPayload {
    MedicationCoverageCheck {
        medication: CodedValue,
        medication_order_ref: Option<FactId>,
        coverage_ref: Option<FactId>,
        checked_at: Timestamp,
        formulary_tier: Option<u8>,
        formulary_status: FormularyStatus,
    },
}

FormularyStatus {
    Covered,
    NotCovered,
    PriorAuthRequired,
    StepTherapyRequired,
    QuantityLimited,
    NotChecked, // no RTBC was performed
}

```

The enclosing Fact records the benefit service’s assertion metadata and cites the RTBC response through `Fact.evidence`. Its `medication_order_ref` is a semantic connection to the order, not a substitute for that evidence. The `MedicationCoverageCheck` represents “what the real-time benefit check said at prescribing.” The `PharmacyCoverageResult` on `PharmacyClaim` represents “what the PBM actually adjudicated at dispensing.” The two often agree; when they diverge, the divergence itself is informative (“the RTBC said covered, but the PBM rejected — something changed between prescribing and pickup”).

Formulary is properly a plan-design property, and the tier value only has meaning in the context of a specific plan's formulary. Modeling the plan formulary itself is out of scope for this document (see Section 14); capturing the tier at observation time is sufficient for patient-facing reasoning.

## 10 Payment sources and methods

The current `Payment` payload and `PayerType` enum treat payments as a small closed set:

```
Payment {
  claim_id:  String,
  amount:    Money,
  paid_by:   PayerType,
}

PayerType {
  PrimaryInsurance,
  SecondaryInsurance,
  Patient,
  Other,
}
```

Real patient payments come from a wider set of sources with different tax and financial-management implications. HSA/FSA payments are pre-tax dollars with reporting requirements. Payment plans are partial payments over time and change what the patient owes. Medical credit lines (CareCredit) are debt with interest terms. Charity care and financial assistance reduce the patient's obligation. Collections activity indicates a payment failure that affects credit. Each of these matters to the patient and to any downstream reasoning about what the patient actually spent on healthcare.

```
Payment {
  claim_id:          String,
  provider_statement_ref: Option<FactId>, // link to statement paid
  amount:            Money,
  paid_by:           PayerType,
  payment_method:    PaymentMethod,
  payment_date:      Date,
  payment_plan_ref:  Option<FactId>,
}

PayerType {
  PrimaryInsurance,
  SecondaryInsurance,
  Medicare,
  Medicaid,
  Patient,
  HSA,
  FSA,
  HRA,
  MedicalCredit { issuer: String }, // e.g. CareCredit
  CharityCare { program: String },
  FinancialAssistance { program: String },
  Collections { agency: String },
}
```

```

    WorkersComp,
    AutoInsurance,
    Other { description: String },
}

PaymentMethod {
    Card,
    Check,
    BankTransfer,
    Cash,
    PaymentPlan,
    Adjustment, // contractual or courtesy write-off
    Other,
}

```

A payment plan itself is not a payment; it is an agreement that produces a sequence of payments. Payment plans need their own payload.

An earlier design assumed the provider always manages the payment plan. Some plans instead use third-party medical financing companies, including CareCredit, PayZen, AccessOne, and ClearBalance. Others are managed by collections agencies, hospital charity programs, or state assistance. The `PaymentPlan` payload uses a `PaymentPlanManager` enum to distinguish these:

```

FactPayload {
    // ...

    PaymentPlan {
        plan_id: String,
        manager: PaymentPlanManager,
        total_amount: Money,
        monthly_amount: Money,
        start_date: Date,
        end_date: Option<Date>,
        interest_rate: Option<Decimal>,
        related_statements: Vec<FactId>, // statements covered by plan
    },
}

PaymentPlanManager {
    Provider { institution: InstitutionId },
    MedicalFinancing { issuer: String }, // e.g. CareCredit, PayZen
    CollectionsAgency { agency: String },
    CharityProgram { program: String }, // hospital or nonprofit
    StateAssistance { program: String },
    Other { description: String },
}

```

Individual `Payment` facts made under a payment plan link back via `payment_plan_ref`. This lets the patient see, on any statement, whether the balance is being paid down under an active agreement, whether it is with a third-party financier accruing interest, or whether it has fallen into collections.

Earlier `PayerType` variants such as `MedicalCredit` and `Collections` conflated two questions. `PayerType` answers *what funded this payment*; `PaymentPlanManager` answers *who administers the*

*agreement*. For a patient paying a CareCredit balance, the Payment Fact records the patient as the funding source, while the related PaymentPlan records CareCredit as the financing manager. The two dimensions are orthogonal.

## 11 FactRelation for billing

Billing Facts currently refer to one another through inline FactId fields. Examples include:

- `Adjudication.claim_id` and `Payment.claim_id`;
- `Appeal.original_decision_id` and `AppealEvent.appeal_ref`;
- `Payment.provider_statement_ref` and `Payment.payment_plan_ref`;
- `PlanAccumulator.coverage_ref` and `PharmacyClaim.medication_order_ref`;
- the order and coverage references on `MedicationCoverageCheck`;
- `ProviderStatement.related_claim` and `PaymentPlan.related_statements`.

These are implicit relations. The existing FEN schema handles clinical relations through `EpisodeRelation` and `NarrativeRelation` — explicit relation types with their own assertion metadata. For consistency, and because billing has genuinely many-to-many relations that inline `FactId` fields cannot represent (one payment covering multiple claims; one claim generating multiple statements over time), a `FactRelation` entity is introduced.

```
FactRelation {
    id: RelationId,
    from_fact: FactId,
    to_fact: FactId,
    relation_type: FactRelationType,
    assertion: AssertionProvenance,
    evidence: Vec<EvidenceRef>,
}

FactRelationType {
    // Billing relations
    ClaimAdjudicatedBy, // Claim -> Adjudication
    ClaimPaidBy, // Claim -> Payment
    StatementBillsClaim, // ProviderStatement -> Claim
    StatementPaidBy, // ProviderStatement -> Payment
    AppealsDecision, // Appeal -> PreAuthDecision or Adjudication
    AppealEventBelongsTo, // AppealEvent -> Appeal
    PaymentPlanCovers, // PaymentPlan -> ProviderStatement
    PaymentUnderPlan, // Payment -> PaymentPlan
    AccumulatorForCoverage, // PlanAccumulator -> Coverage
    PharmacyClaimFulfillsOrder, // PharmacyClaim -> medication-order Fact
    CoverageCheckForOrder, // MedicationCoverageCheck -> medication-order Fact
    CoverageCheckForCoverage, // MedicationCoverageCheck -> Coverage
    PreAuthForService, // PreAuthRequest -> planned service reference
    PreAuthDecisionOnRequest, // PreAuthDecision -> PreAuthRequest
    // ... additional types added as needed
}
```

The direction is part of each relation type’s contract. For example, `StatementBillsClaim` always points from a `ProviderStatement` Fact to a `Claim` Fact, while `ClaimPaidBy` always points from a `Claim` Fact to a `Payment` Fact. Ingestion validates the permitted endpoint types.

`FactRelation` is for domain structure, not ordinary evidentiary support. If prior Facts support an Appeal Fact, the Appeal records them in `Fact.evidence`. A separate “supports appeal” edge would duplicate that canonical evidence relationship.

**Migration of inline `FactId` fields.** Existing inline references are preserved during a transition period. The intent is that ingestion produces both the inline field (for backward compatibility with existing queries) and the corresponding `FactRelation` (for the graph representation). Consumers migrate to querying via relations over time. Once no consumer depends on the inline fields, they are removed in a subsequent schema revision. Fields identified for removal:

- `Adjudication.claim_id` → `ClaimAdjudicatedBy`
- `Payment.claim_id` → `ClaimPaidBy`
- `Payment.provider_statement_ref` → `StatementPaidBy`
- `Payment.payment_plan_ref` → `PaymentUnderPlan`
- `Appeal.original_decision_id` → `AppealsDecision`
- `Appeal.supporting_facts` → `Fact.evidence`
- `AppealEvent.appeal_ref` → `AppealEventBelongsTo`
- `PlanAccumulator.coverage_ref` → `AccumulatorForCoverage`
- `PharmacyClaim.medication_order_ref` → `PharmacyClaimFulfillsOrder`
- `MedicationCoverageCheck.medication_order_ref` → `CoverageCheckForOrder`
- `MedicationCoverageCheck.coverage_ref` → `CoverageCheckForCoverage`
- `ProviderStatement.related_claim` → `StatementBillsClaim`
- `PaymentPlan.related_statements` → `PaymentPlanCovers`

**Advantages.** Relations carry assertion metadata and evidence, so “the payer told us this claim goes with this statement” is distinguishable from “Phoros inferred this relation because the amounts and dates match.” Many-to-many relations are directly representable. The graph becomes uniformly queryable: “what facts relate to this appeal” is a single relation lookup, not a scan across multiple inline fields on multiple fact types. New relation types are added without changing existing fact structures.

**Costs.** More entities to store, and joins are required to reconstruct related facts. The database engine must efficiently query relations by `from_fact` and `to_fact`, which requires appropriate indexing. Ingestion pipelines produce more entities per source event. These costs are the standard tradeoff of graph-oriented modeling and are consistent with the tradeoffs already accepted for `EpisodeRelation` and `NarrativeRelation`.

## 12 Updated FactType enumeration

The `FactType` enum expands to include the new variants:

```
pub enum FactType {
    Measurement,
    Prescription,
    Administration,
    Procedure,
    Diagnosis,
    ImagingResult,
    LabResult,
    Note,
    Encounter,
    Referral,
    Document,
    Coverage,
    Claim,
    Adjudication,
    Payment,
    PreAuthRequest,
    PreAuthDecision,
    Appeal,
    ProviderStatement,
    PlanAccumulator,
    AppealEvent,
    PharmacyClaim,
    MedicationCoverageCheck,
    PaymentPlan,
}
```

The `From<FactPayload>` for `FactType` implementation is updated accordingly. The compiler enforces exhaustiveness, so every new `FactPayload` variant requires a corresponding `FactType` match arm.

## 13 Migration and ingestion considerations

The extensions in this document are additive with respect to the graph model. Existing `Coverage`, `Claim`, `Adjudication`, `Payment`, `PreAuthDecision`, and `Appeal` facts must be migrated to the extended structures.

Migration follows three guarantees: preserve the original representation; do not invent source-authored specificity; and make every extraction, reconstruction, or inference traceable to a source citation or prior `Fact`.

- `Claim.billed_codes` and `Claim.billed_amount` migrate to a single-line `ClaimLine` where the source data does not carry line detail. Ingestion pipelines that receive claim-level data without lines produce a synthetic `ClaimLine` with the aggregate amount and codes; the `line_number` is 1 and the enclosing `Fact` uses `AssertionMethod::Inferred`. Its evidence cites the source record or prior `Fact`, and migration metadata records the reconstruction rule version.

- Existing `Coverage` facts without `effective_from` are migrated using `Fact.occurred_at` as `effective_from`. `effective_through` is set to `None` for the most recent coverage and to the next coverage's `effective_from` for prior coverages, provided they are non-overlapping.
- Existing `PreAuthDecision.reason` strings are preserved in the new `denial_narrative` field. The `denial_codes` vector is left empty. A subsequent enrichment pass may map free-text reasons to CARC/RARC codes, but it creates a separate inferred coding Fact with the original denial Fact as evidence. It does not supersede payer-authored content merely because enrichment became available.
- Existing `Appeal` facts are preserved as-is. The `appeal_id` field is populated from any source-system identifier available. If no such identifier exists, the field remains absent; an internal deterministic key may support implementation but must not be presented as a source identifier. `appeal_level` defaults to `Internal1`. `AppealEvent` facts are not retroactively constructed; the appeal lifecycle is tracked prospectively from the migration date forward.
- Existing `Payment` facts are preserved as-is. The `payment_method` field defaults to `Other` where the source data does not distinguish, and can be enriched during ingestion from provider statement records if those are also ingested.

Ingestion pipelines from payer sources (CARIN Blue Button, Patient Access APIs where available) produce the extended structures directly. Ingestion from provider EHR exports (Epic EHI, generic FHIR) continues to produce medical facts; billing detail from EHR exports is typically limited to claim summaries and is ingested as reconstructed single-line claims per above.

For every path, the original PDF, transaction, API response, or EHI export is registered as a `SourceArtifact`; normalized Facts cite exact passages, records, or rows. Source-provided claim and statement identifiers remain `ExternalRef` values and are not confused with those citations.

## 14 What this does not add

The extensions in this document capture billing and insurance events at patient-facing granularity. They do not add:

- **Full plan design.** A `PlanDesign` object describing formulary contents, service-level coverage rules, network tiers, and benefit-year rules is not included. The minimal `PlanStructure` type on `Coverage` (Section 6.1) captures the plan's structural category (HMO, PPO, HDHP, etc.) and cost-share dimensions (has deductible, has copay, has coinsurance), but not the full plan document. Individual accumulator state (Section 5) is captured, but the plan's benefit schedule is not. If a benefits-intelligence use case emerges, full plan design belongs in a separate contextual model outside `FactPayload`.
- **Vendor and benefit program utilization.** Enrollment in and interaction with employer-sponsored benefit vendors (fertility, mental health, MSK, diabetes management) is not modeled. These are relevant to a benefits-intelligence product but not to a patient-facing health record.
- **Employer or plan-sponsor identity on coverage.** `Coverage` identifies the payer and plan but not the plan sponsor. Adding sponsor identity would enable aggregation across a workforce but would make the FEN record employer-legible in ways that conflict with the patient-owned positioning.

- **Negotiated rate and market pricing context.** Cost transparency comparisons (“what did this employee pay for a knee MRI versus the plan’s negotiated rate at another facility”) require reference pricing data alongside claims. This is out of scope for patient-facing billing defense and belongs in a separate pricing model if pursued.

## 15 Ontological placement summary

| Concern                                                | Placement                                   |
|--------------------------------------------------------|---------------------------------------------|
| Original EOB, statement, export, or transaction        | Immutable <b>SourceArtifact</b> outside FEN |
| Claim, Adjudication, Payment, Appeal, or billing event | FEN Fact payload                            |
| Payer, provider, pharmacy, or PBM identity             | Contextual <b>Institution</b> registry      |
| Claim-to-payment or statement-to-claim connection      | Authored <b>FactRelation</b>                |
| Evidence supporting a billing Fact                     | <b>Fact.evidence</b>                        |
| Source-system identity or correlation key              | <b>Fact.external_refs</b>                   |
| Current appeal state or reconciliation view            | Projection or operational materialization   |

These omissions are deliberate. The extensions in this document strengthen Phoros as a patient-owned health passport with meaningful billing awareness. They do not turn Phoros into a benefits-intelligence platform.